

Agentic Systems Architecture Survey — 2026-07-04

Purpose: Learn-from survey (not a pick-a-tool scan). Each system is evaluated against the 9 component requirements for the Second Brain agent, so we design our system on top of what is *known*, not blind.

Lens: [requirement.md](#) — 9 components + 4 cross-cutting principles (token efficiency, optimum work allocation, local-first, controllability-by-construction).

Method: Three parallel research agents (WebSearch + WebFetch against official docs, GitHub repos, engineering posts, arXiv papers) + direct source-code reading of Kaushik's own systems (CM, Velhari, Velasari settings/skills). All external claims are cited by URL in the Sources section. Kaushik's systems are documented from first-hand code read.

Systems surveyed (11 total):

#	System	Category
1	OpenClaw	OSS local personal agent
2	Hermes Agent (Nous Research)	OSS self-improving agent
3	Claude Code / Agent SDK (Anthropic)	Agentic dev CLI + SDK
4	Letta / MemGPT	Memory-centric agent platform
5	OpenHands (formerly OpenDevin)	Software engineering agent
6	Goose (Block / AAIL)	General-purpose OSS agent
7	CrewAI	Multi-agent orchestration framework
8	Manus AI	Autonomous task completion agent
9	Velasari	Kaushik's evolving persona (reference)
10	Cognitive Memory (CM)	Kaushik's memory system (reference)
11	Velhari	Kaushik's multi-provider orchestrator (reference)

Part 1 — The Matrix: Systems × 9 Components

Maturity marks: **Strong** / **Partial** / **Absent**

System	C1: Persona	C2: Memory	C3: Orchestrator	C4: Connectors	C5: Tools	C6: Skills	C7: Interface	C8: Self- Correction	C9: Observability
OpenClaw	Strong	Strong	Partial	Partial	Strong	Strong	Strong	Strong	Strong
Hermes Agent	Strong	Strong	Strong	Strong	Strong	Strong	Strong	Strong	Strong
Claude Code	Partial	Partial	Partial	Strong	Strong	Partial	Strong	Strong	Partial
Letta / MemGPT	Strong	Strong	Partial	Partial	Partial	Strong	Partial	Partial	Strong
OpenHands	Partial	Partial	Partial	Partial	Strong	Partial	Strong	Strong [†]	Strong
Goose	Absent	Partial	Partial	Partial	Strong	Partial	Strong	Partial	Partial
CrewAI	Partial	Strong	Strong	Strong	Strong	Absent	Partial	Partial	Partial [#]
Manus AI	Absent	Partial	Partial	Partial	Strong	Partial	Partial	Partial	Partial
Velasari	Strong	Strong	Strong	Strong	Strong	Strong	Partial	Strong	Partial

System	C1: Persona	C2: Memory	C3: Orchestrator	C4: Connectors	C5: Tools	C6: Skills	C7: Interface	C8: Self- Correction	C9: Observability
CM (memory organ)	Absent	Strong	Absent	Absent	Strong	Absent	Partial	Absent	Partial
Velhari (orch. organ)	Absent	Absent	Strong	Absent	Strong	Absent	Partial	Absent	Strong

+ OpenHands self-correction is Strong *within-session*; Absent cross-session. ‡ CrewAI observability is Partial for open/local; Strong for commercial AMP.

Part 2 — Per-System Deep Notes

1. OpenClaw

What it is: MIT-licensed self-hosted personal AI assistant. Crossed 100K GitHub stars within a week of launch (Jan 2026). Core design: Gateway daemon (WebSocket, 127.0.0.1:18789) + minimal Pi runtime (4 tools: Read/Write/Edit/Bash) + plugin/skill layer on top. State = human-readable Markdown files in a Git-friendly workspace. Canonical rule: *if it's not written to a file, it doesn't exist*.

C1 — Persona (Strong): `SOUL.md` is slot #1 in every system prompt — the agent literally reads itself into being each session. Identity is a *writable* Markdown file; agents (and skilled users) can modify `SOUL.md` autonomously. `HEARTBEAT.md` scheduler runs a 30-min background consolidation pass that can promote learnings into `SOUL.md`, creating a living identity.

C2 — Memory (Strong): Five-layer: `MEMORY.md` (long-term curated facts), `memory/YYYY-MM-DD.md` (daily episodic logs, today + yesterday auto-loaded), `DREAMS.md` (consolidation diary), SQLite session JSONL, optional vector store (OpenAI/Gemini/Voyage/Ollama/LanceDB/Honcho). *Dreaming* = opt-in background pass that scores short-term signals by recall frequency + query diversity → promotes qualified items to `MEMORY.md`. Hybrid recall: semantic + keyword. Pre-compaction flush saves context before token overflow.

C3 — Orchestrator (Partial): Sub-agents via `sessions_spawn/send/list/history`. Sessions are branching trees, not linear logs — allows diagnostic sub-quests with context rewind. Community frameworks (SWAT, OpenMOSS, Mission Control) add structured committee coordination, but these are third-party. No native task board or heartbeat/zombie detection.

C4 — Connectors (Partial): 21+ messaging platform adapters (WhatsApp, Telegram, Discord, Slack, Signal, iMessage, Teams, Matrix, Feishu, LINE, and more). These are *delivery channels*, not knowledge-ingestion connectors. No native Gmail/Drive/Docs ingestion — must be assembled via skills/tools.

C5 — Tools (Strong): Core Pi: Bash, Read, Write, Edit. Skills register additional tools. Browser automation (A2UI/Canvas), cron jobs, webhooks, MCP protocol, nodes (device connectors). Tool hot-reloading, auto-retry on idempotent failures.

C6 — Skills (Strong): `skills/<skill>/SKILL.md` with three scopes (bundled/managed/workspace). ClawHub registry with 5,700+ community skills. Agent can auto-pull from ClawHub. Self-extension: agent writes code, reloads extensions, tests iteratively. Two-agent adversarial self-improvement: Generator proposes → Evaluator approves/rejects. Correction cascades: `LEARNINGS.md` → `ERRORS.md` → `HEARTBEAT` promotes to `AGENTS.md/SOUL.md`. `agentskills.io` open standard.

C7 — Interface (Strong): 21+ messaging bots via Gateway; Electron desktop app (v0.16.0+, macOS/Linux/Windows); voice interface (macOS/iOS/Android); Canvas (live visual workspace with A2UI).

C8 — Self-Correction (Strong): Session tree branching (diagnostic sub-sessions without polluting main context); `HEARTBEAT` consolidation loop (30-min promotion of `LEARNINGS.md/ERRORS.md`); adversarial Generator/Evaluator with externalized evaluation criteria; `openclaw doctor` auto-resolves 80%+ common runtime issues. Write-ahead checkpointing for crash recovery.

C9 — Observability (Strong): `/verbose`, `/trace`, `/usage`, structured health commands. OTEL integration (v2026.2+ via `diagnostics.otel`). ClawMetry dashboard: tool calls, memory file changes, cron history, sub-agent spawns, session transcripts, per-session cost. ReAct loop exposed as spans.

Key flaws: Dreaming is opt-in (default = memory bloat). No native knowledge-ingestion connectors (email/Drive/docs). Orchestration committee patterns are third-party. Agents can rewrite SOUL.md in emergent ways — identity drift is a documented real-world risk.

Key sources: <https://github.com/openclaw/openclaw> · <https://docs.openclaw.ai/concepts/memory> · <https://softmaxdata.com/blog/deep-dive-into-openclaws-agentic-orchestrate-design-patterns-philosophy-framework-choices/>

2. Hermes Agent (Nous Research)

What it is: MIT-licensed, self-improving AI assistant from Nous Research. Explicit thesis: *the closed learning loop* — every multi-step task triggers automatic skill creation; FTS5 search across all past sessions; 10-turn internal review cycle for persistence decisions. Note: Hermes forked from / succeeded OpenClaw; migration tools exist.

C1 — Persona (Strong): **SOUL.md** (in the Hermes home dir) is slot #1. Multiple built-in personalities (helpful, concise, technical, kawaii, pirate, philosopher, etc.). Custom personalities in **config.yaml**. **USER.md** captures user preferences/interaction history. Multi-profile via separate **HERMES_HOME**. Reasoning toggle: **/reasoning** for extended thinking; **<think>** blocks stripped via context scrubbing. Less autonomously evolutive than OpenClaw — SOUL.md is not auto-rewritten by default.

C2 — Memory (Strong): Three native tiers: **MEMORY.md** (~800 token budget, frozen snapshot at session start for prefix-cache efficiency) + **USER.md** (~500 tokens) + SQLite FTS5 session index (unlimited, ~20ms keyword search, Gemini Flash summarizes snippets). 10-turn internal review nudge. 8 pluggable external providers (Honcho dialectic user modeling, Mem0, Hindsight, Supermemory, and more). Context compression at 50% window threshold via separate compression LLM slot. Skills layer serves as procedural memory.

C3 — Orchestrator (Strong): **AIAgent** single class serves all surfaces. **delegate_task** spawns sub-agents with background fan-out (v0.18.0+, non-blocking). Durable kanban board (v0.13.0+): tasks with status, worker identity, heartbeats; missed heartbeat → task marked suspect → worker reclamation; zombie detection. Mixture-of-Agents (MoA) ensembles selectable as first-class "models" with per-reference-model visible reasoning. **/goal** with completion contracts: agent must produce evidence before claiming done.

C4 — Connectors (Strong): 22 messaging platforms (Telegram, Discord, Slack, WhatsApp Business API, Signal, SMS, Email, Home Assistant, Matrix, DingTalk, Feishu/Lark, WeCom, WeChat, iMessage, Teams, LINE, SimpleX, Webhook adapter). Gmail + Google Drive via OAuth (one auth unlocks full Workspace). MCP bidirectional: client (external servers) and server (expose Hermes via MCP).

C5 — Tools (Strong): 60+ tools: web (web_search, Firecrawl extract), terminal/files (6 backends: local, Docker, SSH, Singularity, Modal, Daytona), browser (Chrome CDP: text + vision + bounding boxes), media (vision, video, image, TTS), agent orchestration, computer use (cua-driver), memory/recall, automation (cron, send_message), RL trajectory generation. Dangerous command approval system + pre-execution scanner.

C6 — Skills (Strong): agentskills.io standard. Progressive disclosure: metadata at startup (~3K tokens), full content on activation. Conditional activation: skills hide/show based on toolset availability. Agent auto-creates skills after 5+ tool call tasks, after error recovery, after user corrections. **skill_manage** tool. Security scanner checks for exfiltration/injection/destructive commands. Multi-source hubs: official, skills-sh, clawhub, github, lobehub. **/learn** command distills skills from any workflow. **/journey** shows skill accumulation timeline.

C7 — Interface (Strong): CLI REPL (Rich + prompt_toolkit); React Ink TUI; Web dashboard (FastAPI); 22-platform messaging gateway (LRU 128 agent instances, 1-hr idle eviction); native Electron desktop (v0.16.0+). **hermes proxy** exposes OpenAI-compatible endpoint (Codex, Aider, Cline can reuse it). Mid-session model switching without history loss.

C8 — Self-Correction (Strong): 5 layers: 10-turn review nudge; trajectory-to-skill pipeline (5+ tool calls → auto-skill); write-time verification (v0.14.0+: file mutation summaries + language-server diagnostics before next turn); **/goal** completion contracts (tests/lint must pass before "done"); **pre_verify** hooks. Self-evolution companion: DSPy + GEPA optimizes skills/prompts against benchmarks (ICLR 2026 Oral Paper).

C9 — Observability (Strong): Structured logs: **agent.log**, **errors.log**, **gateway.log**. Auto-redaction of secrets in debug output. SQLite FTS5 session audit trail. Langfuse integration (one span/turn, one generation/API call, one observation/tool call). OTel via Alibaba Cloud plugin (OTLP, GenAI semantic conventions, per-call token/latency/cost, security anomaly detection). **/journey** memory graph. **hermes status** + **hermes portal info**.

Key flaws: No GDPR-compliant user model deletion (Honcho). Auxiliary model silent degradation when slot is unconfigured. Agent-created skills lack signed provenance/immutable promotion logs. **MEMORY.md** hard-capped at ~800 tokens — intentional (prefix-cache) but limits working knowledge.

Key sources: <https://github.com/NousResearch/hermes-agent> · <https://deepwiki.com/NousResearch/hermes-agent/1.1-architecture-overview> · <https://blakecrosley.com/guides/hermes> · <https://kisztof.medium.com/hermes-agent-review-nous-researchs-self-improving-ai-agent-e72bc244435a>

3. Claude Code / Agent SDK (Anthropic)

What it is: Anthropic's agentic CLI, SDK, and managed platform for software development tasks. Three surfaces sharing the same engine: interactive CLI/IDE/desktop/web, Python/TypeScript Agent SDK, and hosted Managed Agents REST API. Agent loop = context management (CLAUDE.md + MEMORY.md) + hooks (behavioral control at lifecycle events) + tools + MCP.

C1 — Persona (Partial): Identity encoded in CLAUDE.md files at four scope levels (managed/user/project/local). Agent auto-memory in MEMORY.md (Claude-written, first 200 lines or 25KB loaded at session start). No explicit persona block — the agent doesn't autonomously evolve its identity without being instructed. CLAUDE.md is human-authored; MEMORY.md accumulates factual learnings but doesn't reshape reasoning style.

C2 — Memory (Partial): Two systems: CLAUDE.md (human-written, injected as context) + MEMORY.md (agent-written, topic-specific files, local to git repo, not synced). No vector retrieval, no hybrid search, no tiered storage — all memory is context-loaded flat markdown. No episodic/semantic/procedural distinction. Session resumption via JSONL transcripts (**resume: sessionId**). No decay — files accumulate until pruned.

C3 — Orchestrator (Partial): Subagents: spawned via **Agent** tool, own context window, custom system prompt, specific tool allowlist, foreground or background. Results returned to orchestrator. Agent Teams (experimental, v2.1.32+): lead + peer teammates, shared file-based task list with 3 states + dependency blocking, mailbox system (SendMessage), hooks for Teammateldle/TaskCreated/TaskCompleted, plan approval workflow. Explicit: "3–5 teammates proven; beyond that uncharted."

C4 — Connectors (Strong): MCP is the universal connector. 200+ servers in Connectors Directory (July 2025): Gmail, Slack, Google Drive, Notion, Jira, GitHub, Linear, PostgreSQL, Puppeteer/Playwright, filesystem. Three transport types: stdio, HTTP+SSE, Streamable HTTP. MCP can also act as a *channel* — pushing events into Claude's session (Telegram, Discord, webhooks). **@path/to/file** imports pull project docs at session start.

C5 — Tools (Strong): Built-in: Read, Write, Edit, Bash, Glob, Grep, WebSearch, WebFetch, Monitor, AskUserQuestion. MCP tools extend set. Agent SDK: **allowed_tools** allowlist scopes per agent. Custom in-process tools (Python async / TypeScript). All tool calls hook-interceptable at PreToolUse/PostToolUse.

C6 — Skills (Partial): **.claude/skills/*/SKILL.md** with frontmatter (name, description, invocation control, tools allowlist, model, subagent flag). **invokedBy: claude** (auto) or **invokedBy: user** (explicit). **subagent: true** for isolated context. **\$ARGUMENTS** dynamic injection. **once: true** for one-shot sessions. Hooks in skill frontmatter. agentskills.io open standard. **Gap:** no built-in mechanism for agent to auto-package its learnings as skills — this is a community convention ("Learnings Loop"), not an SDK feature.

C7 — Interface (Strong): Five surfaces: Terminal CLI, VS Code/Cursor, JetBrains, Desktop, Web (claude.ai/code). All share CLAUDE.md, settings, MCP servers. Additional gateways: Slack integration, GitHub Actions/GitLab CI/CD, GitHub Code Review, Channels (event push from Telegram/Discord/iMessage/webhooks), Remote Control (continue local session from phone), Dispatch (phone → Desktop task), Routines (cloud-hosted recurring agents), Scheduled tasks.

C8 — Self-Correction (Strong): Hooks system at ~20 lifecycle events (PreToolUse, PostToolUse, PostToolBatch, Stop, SubagentStop, Teammateldle, TaskCompleted, etc.). Hooks can: block action, inject **additionalContext** into context window, return **updatedInput** to modify tool args, return **updatedToolOutput** to modify what agent sees, return **decision: "block"** at Stop to force continuation. This is *deterministic behavioral control* at decision points — independent of model reasoning. Auto memory enables lessons to persist: Claude writes MEMORY.md corrections.

C9 — Observability (Partial): Architecture is "observable if you build the observer." Primitives: JSONL session transcripts + hooks (PreToolUse/PostToolUse log to any backend). No native Step table, no Prometheus metrics, no TTFT histogram. Community implementations (claude-code-hooks-multi-agent-observability) demonstrate real-time dashboards via hooks → HTTP → SQLite → WebSocket → Vue. Reasoning is not externally visible (internal model chain-of-thought).

Key flaws: Memory is flat markdown — no vector retrieval, no semantic search, 25KB cap; not viable for large long-running knowledge bases. Persona doesn't self-evolve. Agent Teams experimental with known limits (no session resumption with in-process teammates, no nested teams). Observability requires custom infrastructure. Context compaction can silently lose nested CLAUDE.md context.

Key sources: <https://code.claude.com/docs/en/overview> · <https://code.claude.com/docs/en/hooks> · <https://code.claude.com/docs/en/agent-teams> · <https://docs.anthropic.com/en/docs/build-with-claude/agent-sdk>

4. Letta / MemGPT

What it is: Platform originating from the MemGPT paper (arXiv:2310.08560, UC Berkeley 2023). Central insight: virtual context management — hierarchical memory tiers (RAM → SSD → disk) give a finite-context LLM unlimited memory. By 2025 evolved into a full stateful-agent platform with a REST API exposing every component of agent state as an addressable resource.

C1 — Persona (Strong): `persona` memory block — always present in context as `<memory_blocks><persona>...</persona></memory_blocks>`. Agent can autonomously rewrite via `core_memory_replace()` and `core_memory_append()`. Letta Code extends this: agents can rewrite persona block, system prompt, and harness configuration over long horizons. V1 agent loop uses native reasoning tokens (not prompt-based thinking tags).

C2 — Memory (Strong): Three-tier: Core Memory (labeled blocks in system prompt, zero retrieval latency, agent-editable, shareable across agents for real-time multi-agent coordination) → Recall Memory (full conversation history in SQL + async vector indexing, hybrid RRF search, filtered by role/date/pattern) → Archival Memory (long-term vector passage store: `SourcePassage` from files + `ArchivalPassage` from agent, pgvector/Turbopuffer/Pinecone backends). Sleep-time agents: dedicated background agents share memory blocks with primary, run during idle periods for compaction/deduplication/summarization. Git-backed memory (Letta Code). **Gap: No decay** — all memories persist until explicitly deleted.

C3 — Orchestrator (Partial): No native orchestration runtime — defers to LangGraph. Shared memory blocks as coordination substrate: when one agent updates a shared block, all agents sharing it see the update immediately (implicit coordination). Sleep-time agents = background orchestration. Letta Code: subagent composition where agents invoke others as specialized extensions. No native parallel-task manager, no committee pattern, no lead/worker hierarchy.

C4 — Connectors (Partial): MCP integration (Streamable HTTP recommended; stdio; server-side). Composio library (7000+ connectors via OAuth in ADE: Gmail, Slack, Google Drive). External data as `SourcePassage` entries in archival memory. No native deep-document ingestion pipeline, no chunking/parsing strategy.

C5 — Tools (Partial): Built-in memory tools (`core_memory_append/replace`, `archival_memory_insert/search`, `conversation_search`) executed by `LettaCoreToolExecutor`. Server tools: Python functions in Letta's sandbox; ADE Python editor with mock testing. MCP tools route to external servers. Human-in-the-loop tool type. Narrower than Claude Code — primarily memory-and-API, not full agentic workstation.

C6 — Skills (Strong): Mountable directories with `SKILL.md` plus resources. Hierarchically scoped: global, project, agent-specific (MemFS/git). Agent selects/mounts/uses/unmounts dynamically — tokens only for mounted skills. December 2025: Skill Learning — agents package learned solutions as new skills from experience. November 2025: Context-Bench benchmark suite for skill discovery.

C7 — Interface (Partial): Agent Development Environment (ADE) at chat.letta.com — three simulation modes: Debug (full context window + memory block inspection + tool call trace), Interactive, Simple. REST API on port 8283. No native Slack/email gateway. ADE's context window visualization is genuinely exceptional — the most inspectable agent UI in this survey.

C8 — Self-Correction (Partial): Sleep-time agents rewrite messy memory blocks, deduplicate, consolidate. Agent self-editing during any turn. V1 native reasoning tokens allow pre-action reflection. No hook-like system for automated correction injection at decision points. Five concrete failure modes documented (empty blocks, runaway growth, archival junk drawers, semantic search misses, cross-user pollution) — diagnosable but not automatically corrected.

C9 — Observability (Strong): Every LLM inference generates a `Step` record: provider metadata, model, token usage (prompt/completion/cached/reasoning), execution status, stop reason, LLM latency (ns), tool execution time (ns), total wall-clock (ns), `trace_id`, `request_id`. Stored in PostgreSQL, exposed at `/v1/steps/{step_id}/trace`. Raw provider traces in PostgreSQL/ClickHouse/external sidecar via `TelemetryManager`. OTel: TTFT histogram, LLM execution time, step execution time, message counters, tool execution counters, async task gauges. Git-backed memory = version-controlled audit of all memory mutations.

Key flaws: No memory decay — long-running agents accumulate stale facts silently. Concurrent block writes are destructive (optimistic locking helps but doesn't prevent loss). Orchestration requires external scaffolding (LangGraph). Tool prompt discipline not automatic. Skill self-authoring is recent (Dec 2025), production maturity unproven.

Key sources: <https://arxiv.org/abs/2310.08560> · <https://github.com/letta-ai/letta> · <https://docs.letta.com/guides/agents/memory-blocks> · <https://deepwiki.com/letta-ai/letta/3-memory-system> · <https://www.letta.com/blog/letta-v1-agent>

5. OpenHands (formerly OpenDevin)

What it is: Software-engineering-focused AI agent platform. V0 used `AgentController` + pub/sub `EventStream` (fragile, async ordering bugs). V1 (Nov 2025): OpenHands Software Agent SDK — four packages. Agent is a *stateless pure function* mapping conversation history to next action; only `ConversationState` mutates, by appending to an immutable event log. Execution model: **CodeAct** — agents emit Python/bash/browser code rather than bespoke JSON tool schemas. Research shows CodeAct achieves 55–87% reduction in input tokens vs. traditional tool-calling. LLM integration via LiteLLM (100+ providers). `RouterLLM` routes vision-heavy messages to vision-capable models.

C1 — Persona (Partial): System prompt encodes four-phase methodology: Exploration → Analysis → Implementation → Verification. `AgentThinkTool` provides explicit reasoning steps. `ReasoningLLM` wraps Claude Extended Thinking as first-class event properties. Skills can inject persona-specific instructions conditionally. No persistent persona object — identity from system prompt composition.

C2 — Memory (Partial): Within-session: append-only `EventLog` (every action + observation persisted as JSON). `LLMSummarizingCondenser`: when log exceeds 80 events, preserves first 4 + last ~40, summarizes middle via auxiliary LLM, emits `CondensationEvent` (~2x API cost reduction). No cross-session episodic memory — new conversation starts fresh. `RecallAction` type exists but requires operator wiring of MCP/skills.

C3 — Orchestrator (Partial): `DelegateTool` spawns up to 5 sub-agents as `threading.Thread` instances (configurable `max_children`), parent blocks until all complete, results aggregate into `DelegateObservation`. `TaskToolSet` manages sequential sub-agent tasks with pause/resume + persistent state. Sub-agents inherit parent LLM config, share workspace, independent `EventLogs`. No manager pattern, no hierarchical orchestration primitive.

C4 — Connectors (Partial): Native: GitHub (issue creation, PR submission), GitLab, Bitbucket, Slack (cloud-only). `AGENTS.md`/`CLAUDE.md` convention for repo-scoped knowledge injection. MCP (`MCPAction`) for additional connectors. No first-party email/Drive/document ingestion.

C5 — Tools (Strong): Strongly typed Action/Observation/Executor pattern (Pydantic). Core actions: `CmdRunAction` (shell), `IPythonRunCellAction` (Jupyter kernel), `FileReadAction/WriteAction/EditAction`, `BrowseURLAction` (Playwright), `MessageAction`, `AgentThinkAction`, `MCPAction`. MCP schemas translate to Action models. OAuth for local + remote MCP. `NonNativeToolCallingMixin` for prompt-and-parse fallback.

C6 — Skills (Partial): Markdown files with YAML frontmatter. Activation: always-on or `KeywordTrigger/TaskTrigger`. Inline shell commands via backtick syntax for dynamic injection. Skills can spawn MCP server processes on activation. `AGENTS.md` in repo roots auto-loaded. `register_agent()` defines sub-agent specifications as skills. No self-authoring.

C7 — Interface (Strong): React web UI at localhost:3000. FastAPI REST + WebSocket. OpenHands Cloud API (launched Nov 2025). Slack bot (cloud-only). VS Code extension. Agent Canvas: connect frontend to multiple Agent Servers on different hosts. Visual workspace: VS Code embedded, VNC, browser.

C8 — Self-Correction (Strong, in-session): Three-layer: (1) Observation-driven — every command's stderr/exit code flows directly into next prompt as typed Observation. (2) `StuckDetector` — runs every step on `EventLog`, detects 5 semantic pathology patterns (repeating action-observation pairs ≥4, alternating error pairs ≥3, monologue ≥3, ping-pong cycles ≥6, repeated context-window errors). (3) Error classification into 6 types with type-specific recovery guidance. (4) `LLMSummarizingCondenser` prevents context thrash. **No cross-session behavioral learning.**

C9 — Observability (Strong): Immutable `EventLog` IS the audit trail — every event has id, source, timestamp; LLM reasoning/thinking blocks included. Read-only auxiliary services consume the log, never mutate it. `SecurityAnalyzer` scores every action LOW/MEDIUM/HIGH/UNKNOWN before execution. `ConfirmationPolicy` gates execution. `SecretRegistry` masks API secrets in output and encrypts at rest. LiteLLM tracks token counts, costs, latency.

Key flaws: Software-dev tunnel vision — knowledge intake (PDFs, emails) requires manual MCP plumbing. No cross-session memory. Sub-agent orchestration is primitive (no manager pattern). **StuckDetector** threshold tuning is delicate.

Key sources: <https://arxiv.org/html/2511.03690v1> · <https://deepwiki.com/All-Hands-AI/OpenHands> · <https://docs.openhands.dev/sdk/arch/overview>

6. Goose (Block / AAILF)

What it is: General-purpose, model-agnostic agent built in Rust as a Cargo workspace. Released by Block, transferred to Linux Foundation AAILF (2026). Core: **Agent::reply()** async loop — prepares context from session history, calls LLM + available tools, dispatches via **ExtensionManager**, handles approvals, iterates. Sessions persist in SQLite via **sqlx**. 25+ LLM providers via **ProviderRegistry**, ~1700 canonical models. Deeply MCP-centric (claims earliest/deepest MCP adoption).

C1 — Persona (Absent): No persona abstraction. Identity emerges from configuration profiles (**config.yaml**), extension loading, and Recipe parameterization. No persistent identity object that evolves.

C2 — Memory (Partial): Within-session: SQLite session history, context compaction at 80% window via summarization, named sessions, resume by ID, fork, export. Cross-session: MCP-based **MemoryServer** (**remember_memory**, **retrieve_memories**) — flat-file store at **.goose/memory/** or global memory dir. All saved memories load at session start via system prompt injection. **chatrecall** platform extension searches past conversations. **Gap:** No decay, no deduplication/consolidation, no importance scoring, no semantic search — category-based retrieval only.

C3 — Orchestrator (Partial): **summon** extension delegates to sub-agents with scoped extensions. Proposed **AgentManager** (Discussion #4389, in implementation): per-session agent isolation with **ExecutionMode: Interactive|Background|SubTask**. Recipes (YAML + minijinja): reusable parameterized workflows with parallel execution mode, automated retry, shell command validation. **ProviderRegistry** with lead/worker routing (different models for different task costs). No committee/crew pattern.

C4 — Connectors (Partial): Extension-based: any MCP server is a potential connector. First-party: Developer (filesystem), Computer Controller (screen/browser), Memory. Third-party MCP: Slack, GitHub, Google Drive, email, Notion. Graphlit MCP provides structured Slack/Gmail ingest. No first-party knowledge-intake connectors.

C5 — Tools (Strong): **ExtensionManager** orchestrates MCP extensions across 4 transport types (stdio, HTTP/SSE, in-process Rust, platform). 70+ official extensions. Permission modes: auto/approve/smart_approve/chat_only. Built-in: shell, file read/write/edit, AST analysis (tree-sitter), web scraping, PDF/DOCX extraction, screen automation (macOS/Windows/Linux), scripts (Shell/Ruby/Batch/PowerShell).

C6 — Skills (Partial): **skills** platform extension loads static YAML/Markdown instruction sets. Recipes as packaged procedures. No self-authoring, no accumulating skills from experience.

C7 — Interface (Strong): CLI (goose REPL + headless), Desktop app (Electron + React, manages **goosed**), web server mode. **goosed** server via ACP (JSON-RPC 2.0 over POST **/acp**): session management, streaming, tool execution with permission flows. Slack bot proposed (not shipped first-party).

C8 — Self-Correction (Partial): Tool result observation (stdout/stderr into next LLM call). Recipe validation (shell command outcome checks, automated retry). Context compaction. No stuck detection, no error taxonomy, no lessons-learned capture.

C9 — Observability (Partial): OTel (OTLP) in **crates/goose/src/otel/otlp.rs**. Langfuse integration for session-level LLM traces. Permission pipeline audit log. Prompt injection detection (configurable sensitivity). Malware scanning via OSV database for external extensions. No immutable reasoning trace as audit artifact.

Key flaws: Shared-Agent session bug (one Agent for all sessions — cross-session interference; AgentManager in progress). Memory is a flat KV store. Rust language barrier for extension authoring. No structured multi-agent orchestration. AAILF governance transition risk.

Key sources: <https://deepwiki.com/block/goose> · <https://github.com/block/goose> · <https://goose-docs.ai/docs/mcp/memory-mcp/> · <https://langfuse.com/docs/integrations/goose>

7. CrewAI

What it is: Python framework for orchestrating role-playing, collaborative AI agent crews. Core triad: **Agent** (persona with role/goal/backstory + LLM + tools + memory) + **Task** (unit of work with context dependencies) + **Crew** (orchestrator). Above Crews: **Flows** — event-driven workflow layer sequencing multiple Crews using `@start/@listen/@router` decorators with `or_()/and_()` combinators. Multi-LLM: primary LLM for reasoning, separate `function_calling_llm` for tool invocation (cheaper), `manager_llm` for hierarchical coordination.

C1 — Persona (Partial): Agents have Role (function/expertise), Goal (individual objective), Backstory (personality/context) — richest explicit persona model of the surveyed frameworks. `reasoning=True` enables planning cycles before task execution. JSONC agent definitions in `agents/<name>.jsonc` (version-controlled). **Gap:** Persona is static per crew definition — does not adapt across sessions from outcomes. No mechanism for agent to update its own backstory.

C2 — Memory (Strong): Current unified system (LanceDB): hierarchical scope paths, composite scoring (`semantic_weight × similarity + recency_weight × decay + importance_weight × importance`, `decay = 0.5^(age_days/half_life_days)`), LLM-guided consolidation on save (`similarity ≥ 0.85` → LLM decides keep/update/delete/insert), shallow recall (vector, ~200ms) vs. deep recall (multi-step LLM-guided + parallel vector + recursive exploration), smart LLM skip (queries <200 chars bypass LLM), background write drain on shutdown. Legacy system: ChromaDB (short-term/entity/RAG) + SQLite3 (long-term). Knowledge Sources: PDF/CSV/Excel/JSON/text/URL vectorized via ChromaDB at crew init.

C3 — Orchestrator (Strong): Sequential or Hierarchical process types. Hierarchical: manager agent (LLM or custom) delegates, validates, coordinates. Flows layer: event-driven DAG, conditional branching via `@router`, `@human_feedback` gates, flow state persistence via `@persist` (SQLite), resume/fork from snapshots. `AgentPlanner` (`planning=True`): planning LLM generates per-task plans added to descriptions before each iteration. Each agent can use a different LLM.

C4 — Connectors (Strong): Enterprise OAuth catalog (CrewAI AMP): Gmail, Slack, Microsoft/Teams/Office365, Jira, ClickUp, Asana, Notion, Linear, GitHub, Salesforce, HubSpot, Zendesk, Google Sheets, Google Calendar, Stripe, Shopify, Box. MCP integration (`crewai-tools[mcp]`). Knowledge Sources: PDF/CSV/Excel/JSON/text/URL vectorized via ChromaDB.

C5 — Tools (Strong): 30+ built-in (web search, scraping, file ops, code analysis, data analysis). `@tool` decorator or `BaseTool` subclass with Pydantic I/O schemas. Tool caching with custom `cache_function`. LangChain tool compatibility. MCP integration.

C6 — Skills (Absent): No skills primitive. Task definitions (reusable task specs) and Knowledge Sources partially substitute but lack the trigger-activated, self-authored, accumulating capability that "skills" implies.

C7 — Interface (Partial): Python library — no built-in UI. Options: AG-UI protocol (CopilotKit) for React frontends, community libraries, Panel/Taipy, custom stacks. CrewAI AMP (commercial) = hosted dashboard. `crewai run` CLI. No first-party chat channel integration.

C8 — Self-Correction (Partial): Retry logic (`max_retry_limit` default 2). Iteration cap (`max_iter` default 20). Context window management (`respect_context_window=True`). `step_callback` fires after each agent step. Memory-based reflection (importance scoring + decay = implicit representation of what mattered). External critic pattern: dedicated reviewer agent in crew — user-configured, not automatic. No stuck detection.

C9 — Observability (Partial): Built-in tracing via `tracing=True`: agent reasoning/decisions, task timelines, tool invocations + results, LLM interactions, performance metrics. → CrewAI AMP at app.crewai.com. OTEL support (SigNoz integration confirmed). Datadog integration. Step/task/kickoff callbacks. Enterprise AMP: RBAC + immutable audit trails + IAM. **Gap:** reasoning trace in cloud-hosted commercial platform, not open/local replayable event log.

Key flaws: Two documented memory architectures (ChromaDB/SQLite vs. LanceDB unified) — documentation confusion. No skills system. UI orphaned to third parties. Commercial creep (key operational features require AMP). Persona is static. Self-correction requires manual critic-agent design.

Key sources: <https://docs.crewai.com/concepts/memory> · <https://docs.crewai.com/concepts/crews> · <https://docs.crewai.com/concepts/flows> · <https://sparkco.ai/blog/deep-dive-into-crewai-memory-systems>

8. Manus AI

What it is: Commercial, closed-source autonomous general-purpose agent by Monica.im; acquired by Meta late 2025 for >\$2B. Goal: "bridge mind to hand" — transform high-level goals into completed deliverables (PowerPoint, PDF, websites, dashboards) rather than conversation turns. Average task: 30–50 steps, 100:1 input-to-output token ratio, executed in cloud-hosted Ubuntu Linux

VM. Architecture: **contextual state machine** with CodeAct (Python code as action primitives). Explicitly rejects fixed Planner/Executor/Verifier roles — "less structure, more intelligence."

C1 — Persona (Absent): No user-configurable SOUL.md equivalent. Identity = foundation model alignment + RLHF + "personal logic system" (user teaches preferred behaviors in natural language, stored for future sessions). No structured identity file, no writable persona.

C2 — Memory (Partial): Four tiers: Event stream (in-session chronological log); File system as extended memory (intermediate results as VM files, KV-cache prefix optimization for 10x cost reduction); **Todo.md** (continuously rewritten goal tracker to keep objectives in recent attention window); Knowledge module (user preferences + uploaded PDFs/spreadsheets, persists cross-session). No FTS/vector recall exposed. No explicit episodic search.

C3 — Orchestrator (Partial): Foundation-model-delegated decomposition. Contextual state machine with logit masking (enforces valid action sequences, one action per iteration for rollback cleanliness). Parallel complex projects: multiple sub-agents in separate VM environments coordinated by high-level orchestrator. Planner module injects ordered plan into context, updated as new information arrives. No durable task board, no heartbeat/zombie detection.

C4 — Connectors (Partial): MCP-protocol connectors: Gmail, Google Calendar, Google Drive, Notion, GitHub, HubSpot, Stripe, Slack. Connector Recommendations: detects missing connectors for active tasks. Browser Operator: persistent login sessions for hundreds of sequential web actions without re-authentication. Connector coverage is real but shallower than Hermes (no SMS, Home Assistant, Matrix, messaging-platform breadth).

C5 — Tools (Strong): 27 abstracted VM-level tools: browser_navigate/click/fill/screenshot, shell_exec, file_read/write, Python interpreter, web server deployment, Node.js executor. Product tools: AI Slides, AI Design, AI Image Generator, Browser Operator, Wide Research, Mail Manus. CodeAct: model generates Python that runs in VM. Multi-modal browser input: viewport text + screenshot + screenshot with bounding boxes simultaneously.

C6 — Skills (Partial): agentskills.io standard. Three-level progressive disclosure (L1 metadata ~100 tokens; L2 SKILL.md <5K tokens; L3 scripts on-demand). Creation: "Build with Manus" packages successful workflows automatically; upload local .zip/.skill/folder; GitHub import; official library. No conditional activation gates, no security scanner, no in-product hub (marketplace = future).

C7 — Interface (Partial): Web app (manus.im). "Manus's Computer" screen-mirror shows real-time VM activity. Shareable replay URLs. Slack DM connector as conversational interface. No self-hosted option, no desktop app, no CLI, no TUI.

C8 — Self-Correction (Partial): Error traces preserved in context (failed actions remain visible → agent updates beliefs implicitly). Context-aware replanning (plan injected, updated when obstacles arise). Persistent browser sessions survive errors for retry. Verification agent reviews outcomes. No adversarial Generator/Evaluator, no trajectory-to-skill pipeline.

C9 — Observability (Partial): Screen mirror + shareable replay URL = intuitive for end-users. Session logs capture decision process. **Gaps:** No structured span/trace export, no OTel integration documented, no reasoning trace exposure (internal thinking not shown), no signed audit trail. Team explicitly acknowledges "black box" nature.

Key flaws: Closed-source, cloud-only — structural blocker for privacy-sensitive Second Brain. No persistent configurable identity. No cross-session learning pipeline. Anti-structure philosophy creates accountability gaps. Black-box reasoning by design. Connector depth is shallow. Meta acquisition = roadmap uncertainty.

Key sources: <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus> · <https://arxiv.org/html/2505.02024v1> · <https://manus.im/docs/integrations/mcp-connectors>

9. Velasari (Kaushik's reference system)

What it is: Claude Code itself, configured as a persistent, evolving agent persona for Kaushik. Not a separate codebase — Velasari is the configured environment: CLAUDE.md global identity, skills/plugins for packaged capabilities, Cognitive Memory MCP for persistent memory, Velhari via Agent tool for multi-provider orchestration, and a rich MCP connector ecosystem.

C1 — Persona (Strong): Global CLAUDE.md encodes the "Velasari" identity, operating philosophy (Antenna Principle, anti-patterns to resist, spec-driven development, tracking discipline). Evolves by Kaushik updating CLAUDE.md; agent can write MEMORY.md learnings. Cross-session identity is stable and richly specified. Cognitive Memory stores identity-type memories with 365-day initial stability (highest decay resistance of any type).

C2 — Memory (Strong): Cognitive Memory MCP (separate service, see System 10). CM provides true episodic/semantic/procedural/identity/person memory with FSRS decay, multi-strategy retrieval, and consolidation. This is a purpose-built memory organ that Velasari invokes via MCP tools. Also: Claude Code's own MEMORY.md for lighter-weight session notes.

C3 — Orchestrator (Strong): Velhari invoked via Agent tool (or via its REST/NATS API). Supports Claude, OpenAI Codex, and local Ollama workers in parallel. Per-worker budget control, guardrails, and monitor windows. NATS messaging bus for event-driven coordination. Distributed mode with heartbeat registry supports multiple machines.

C4 — Connectors (Strong): Configured MCP servers: Gmail (search/read/draft/label), Slack (read/send/search/canvas), Google Drive (read/search/create/copy), Notion, Canva, Monday.com, Puppeteer (browser automation), Taskyn PM. These cover Kaushik's full work surface.

C5 — Tools (Strong): Full Claude Code built-in toolset (Read, Write, Edit, Bash, Glob, Grep, WebSearch, WebFetch, Agent, Monitor) + all configured MCP tools. Agent tool spawns subagents (Explore, Plan, general-purpose, statusline-setup). Skills are callable as slash commands.

C6 — Skills (Strong): e-spec plugin provides full spec-driven development workflow (/spec, /requirement, /design, /implement, /dryrun-*). Additional skills: update-config, keybindings-help, simplify, loop, schedule, claude-api. Skills follow agentskills.io standard. Packaged reusable capabilities that accumulate over time via APEX plugin marketplace.

C7 — Interface (Partial): Currently Claude Code surfaces only (terminal, VS Code, desktop). No custom web app or Slack bot yet — those are targets for the Second Brain project. Remote Control and Dispatch features available.

C8 — Self-Correction (Strong): Claude Code hooks system for behavioral control at lifecycle events. CLAUDE.md anti-patterns table (Lessons Learned section) encodes hard-won behavioral rules that override model defaults. CM used to store and recall past lessons at relevant decision points. autoDream and consolidation for periodic memory health.

C9 — Observability (Partial): JSONL session transcripts. Taskyn PM for work tracking. CM health reports and version history. No custom observability stack — would need hook-based logging to be built.

10. Cognitive Memory / CM (Kaushik's memory organ)

What it is: MIT-licensed biologically-inspired cognitive memory MCP server. Not an agent — a specialized memory service that agents (primarily Velasari) consume via 14 MCP tools. Built with Python + SurrealDB (embedded) + sentence-transformers (all-MiniLM-L6-v2, 384d). [Source code read directly for this survey: engine.py, retrieval.py, decay.py, consolidation.py, models.py]

C1 — Persona (Absent): Component, not agent. No persona.

C2 — Memory (Strong — best-in-class across all 11 systems):

- **6 memory types with differential decay:** working ($S_0=0.04$, ~1hr), episodic ($S_0=2$, ~2 days), semantic ($S_0=14$, ~2 weeks), procedural ($S_0=60$, ~2 months), identity ($S_0=365$, ~1 year), person ($S_0=90$, ~3 months)
- **FSRS-inspired decay formula:** $R(t) = e^{(-t / 9S)}$ — pure on-the-fly computation from last_accessed + stability
- **Reinforcement on access:** $S_{\text{new}} = S * (1 + \text{growth_factor} * (1 - R))$ — low-R memories get bigger boost on retrieval
- **Multi-strategy retrieval (Phase 1, concurrent asyncio.gather):** HNSW semantic search + BM25 FTS keyword + temporal recency (30-day half-life)
- **RRF fusion:** Weighted Reciprocal Rank Fusion (semantic 1.0, keyword 0.7, temporal 0.3)
- **Phase 2 — Graph traversal:** From top-5 Phase 1 seeds, bulk neighbor fetch, RRF with graph weight 0.5
- **Decay-weighted reranking:** $\text{score} = \text{rrf_score} * R^{\text{decay_influence}} + \text{supersede penalty}$
- **Spreading activation:** Retrieving a memory boosts neighbors' stability ($S_{\text{new}} = S * (1 + \text{boost})$), boost decays by depth via $\text{spread_factor}^{(\text{depth}-1)}$
- **Contradiction detection:** High-similarity pair + negation signals → CONTRADICTS edge
- **Auto-linking:** New memories linked to similar existing ones (cosine ≥ 0.75) via RELATES_TO edges
- **Consolidation pipeline:** Decay update → promotion pass (working→episodic→semantic/procedural by access count + importance + relationship count + content patterns) → archive pass ($R < \text{threshold}$) → cluster-and-merge (cosine ≥ 0.90)
- **Version history:** Every update creates a snapshot
- **8 relationship types:** causes, follows, contradicts, supports, relates_to, supersedes, part_of, describes

- **Storage:** SurrealDB embedded (HNSW vectors, BM25 FTS, graph edges). No external DB dependency.
- **14 MCP tools:** `memory_store`, `memory_recall`, `memory_get`, `memory_update`, `memory_relate`, `memory_related`, `memory_unrelate`, `memory_list`, `memory_archive`, `memory_restore`, `memory_delete`, `memory_stats`, `memory Consolidate`, `memory_config`
- **Windows service:** Task Scheduler background service with auto-restart (3 attempts, 1 min apart)
- **Backup/restore:** NDJSON logical export, 3-tier retention (7 daily / 4 weekly / 6 monthly), atomic rename rollback, pre-dream backup requirement

C3–C9: Absent/Partial as component — CM is a memory organ, not an orchestrator or gateway.

Key architectural lesson: CM's FSRS decay + spreading activation + consolidation pipeline is the most biologically accurate memory architecture in this survey. The gap: CM doesn't implement a **semantic entity/relationship knowledge graph** over ingested documents — it has relationship edges between memories, but not an explicit named-entity graph over connector-ingested content. That distinction matters for a Second Brain ingesting structured knowledge from email, Slack, and documents.

11. Velhari (Kaushik's orchestrator organ)

What it is: Multi-provider worker orchestrator. Runs as a Windows service (Task Scheduler). Exposes REST API. Workers communicate via NATS messaging bus. Supports four worker kinds: Claude (via `claude-agent-sdk`), Codex (OpenAI), Local (Ollama via HTTP), Command (shell subprocess). [Source code read directly: `orchestrator.py`, `guardrails.py`, `models.py`]

C1 — Persona (Absent): Orchestrator only. No identity.

C2 — Memory (Absent): Orchestration layer. Workers have their own memory (Claude workers use `CLAUDE.md/CM`).

C3 — Orchestrator (Strong):

- **Multi-provider workers:** Claude (`ClaudeSDKClient` with streaming, budget control, resume), Codex (codex CLI subprocess with sandbox + approval-policy), Local (Ollama httpx probe + structured Pydantic output schema), Command (shell subprocess with timeout + escalation)
- **Worker lifecycle:** `STARTING` → `RUNNING` → `COMPLETED/FAILED/STOPPED`. Per-worker `asyncio.Queue` for control messages. `asyncio.Task` background loops.
- **NATS messaging:** `workers.{id}.status`, `.output`, `.error`, `.completed`, `.control` subjects. JSON-encoded payloads.
- **Per-worker console monitor windows:** Subprocess with `CREATE_NEW_CONSOLE` (observability-by-construction for development).
- **Distributed mode:** Heartbeat registry (configurable interval, default 60s), command listener, project directory scanning, remote launch/list/stop via NATS ack pattern.
- **Follow-up queries:** After `ResultMessage`, worker waits up to 30s for a control message (`query/stop/set_model`) enabling multi-turn worker sessions.
- **Budget control:** `max_budget_usd` per Claude worker.

C5 — Tools / Guardrails (Strong): Per-worker guardrail system (`make_guardrail` factory): Rule 1: Bash command inspection (blocked_commands regex patterns), Rule 2: File path scoping (Edit/Write/Read must be within worker CWD), Rule 3: Blocked path prefixes. Every tool decision published to `sys.audit` NATS subject with timestamp, tool name, tool input summary, decision, reason.

C9 — Observability (Strong): Every tool decision published to `sys.audit`. Worker output streamed to NATS `workers.{id}.output`. Status changes published. Console monitor windows provide real-time operator visibility. Heartbeat registry for distributed instance health. Worker `output_buffer` (deque maxlen=100) for recent output access.

Part 3 — Per-Component Synthesis

C1: Evolving Agent Persona / Thinking Core

State of the art: OpenClaw's `SOUL.md` + `HEARTBEAT.md` + adversarial self-extension is the most complete approach: writable identity file that the agent reads into being each session, a scheduler that promotes learnings into identity, and an adversarial pattern for safe evolution. Hermes matches it. Letta's persona block with v1 native reasoning tokens adds self-editing from within reasoning.

Best implementations: OpenClaw / Hermes (tied) — writable SOUL.md that autonomously evolves. Letta second for the persona-as-typed-block concept.

Known pitfalls: (1) Identity drift: if the agent can rewrite its own SOUL.md, belief injection via scripts is a documented real-world attack. (2) Static persona (CrewAI, Manus, Goose) defeats the "Second Brain" purpose. (3) Flat-file identity (Claude Code's CLAUDE.md) works but requires human intent to evolve.

BORROW THIS / FIX THAT: Separate identity into an explicit writable SOUL.md loaded as slot #1 of every session prompt. Run a scheduled consolidation pass that scores recent learnings and promotes qualified items. Gate self-modification via an adversarial evaluator (Generator proposes, Evaluator approves against externalized criteria). Honor local-first — keep SOUL.md in local filesystem, Git-versioned. Define a schema for identity sections (beliefs, preferences, communication style, known skills, relationships) and constrain edits to those sections only.

C2: Memory

State of the art: Cognitive Memory (CM) is the most biologically rigorous for *individual* memory management: FSRS decay, 6 typed categories, multi-strategy retrieval (HNSW + BM25 + temporal + graph) fused via RRF, spreading activation, contradiction detection, consolidation pipeline. Letta's three-tier architecture (core/recall/archival) with shared memory blocks is the best for *multi-agent* shared memory. CrewAI's LanceDB unified system with composite scoring and LLM-guided consolidation is the closest open-source framework approximation. Hermes's FTS5 frozen snapshot is the best for cross-session retrieval *speed* (~20ms) with prefix-cache efficiency.

Best implementations: CM for individual management. Letta for multi-agent shared memory + observability. Hermes for retrieval speed + cache efficiency.

Known pitfalls: (1) No decay (Letta, Claude Code, Goose) — stale facts accumulate. (2) Flat-file memory (Claude Code, Goose) — no semantic search, degrades with scale. (3) Memory without entity graph — no system builds an explicit named-entity/relationship graph over ingested content. (4) Memory without provenance — most systems don't track the source of a memory, making correction/audit hard.

BORROW THIS / FIX THAT: Reuse CM's FSRS decay + 6 types + differential stabilities. Keep CM's two-phase retrieval (concurrent semantic + keyword + temporal → RRF → graph traversal → decay reranking + spreading activation). Keep CM's consolidation pipeline. Add what CM lacks: a named-entity knowledge graph over connector-ingested content (email → entities → relationships). CM's consolidation should run on a heartbeat schedule automatically, not require explicit invocation.

C3: Orchestrator (Multi-Provider Consortium)

State of the art: Velhari is the only system with true *multi-provider* orchestration (Claude + Codex + local Ollama in the same worker pool with guardrails). CrewAI's Flows + hierarchical crew is the best *multi-agent* orchestration expressiveness (event-driven DAG, conditional branching, state persistence). Hermes's durable task board with heartbeat/zombie detection is the best *reliability* model.

Best implementations: Velhari for multi-provider diversity. CrewAI for orchestration expressiveness. Hermes for production reliability (zombie detection, completion contracts).

Known pitfalls: (1) Shared-Agent bug (Goose v1) — session isolation must be explicit. (2) No committee/consensus pattern — MoA ensembles (Hermes only) give diverse perspectives; no other system implements the "committee" use-case. (3) Orchestration without audit. (4) No budget-aware routing by capability.

BORROW THIS / FIX THAT: Velhari's multi-provider worker pool with per-worker budget and guardrails. Hermes's durable kanban board with heartbeat (missed heartbeat → suspect → reclaim). CrewAI's @router conditional branching. OpenHands' DelegateTool for bounded parallel fan-out. Build the MoA committee pattern: submit the same prompt to N providers, aggregate via synthesis agent. Add optimum-work-allocation routing: classify task complexity → route to cheapest capable model (local Ollama for simple, Codex for code, Claude for reasoning).

C4: Connectors (Knowledge Intake)

State of the art: Hermes has the widest connector coverage (22 messaging platforms + Gmail/Drive OAuth + MCP bidirectional). CrewAI AMP has the widest enterprise OAuth catalog (16+ business apps). Claude Code's MCP ecosystem (200+ servers) has the widest community-driven coverage. No system has a best-in-class *unstructured document ingestion* pipeline with chunking, entity extraction, and graph construction.

Best implementations: Hermes for channel + knowledge connectors combined. Claude Code / MCP ecosystem for extensibility. CrewAI AMP for enterprise CRM/ticketing coverage.

Known pitfalls: (1) Conflating delivery channels with knowledge-ingestion connectors. (2) No built-in document parsing + chunking + entity extraction pipeline. (3) OAuth sprawl — multiple auth flows create friction and security surface.

BORROW THIS / FIX THAT: Hermes's OAuth pattern (one auth unlocks full Gmail Workspace). MCP as the connector protocol. Design the connector layer as *ingestion-first*: every connector has two modes — read/ingest (feeds CM) and write/act (sends to external system). Local-first: ingest to CM on-prem; act to external via network. Treat incoming connector events as memory ingestion triggers (Slack message → classify → store to CM with type + tags + provenance). Build what's missing: document ingestion pipeline that chunks, embeds, extracts named entities, and stores to CM as semantic/episodic memories with source provenance.

C5: Tools (The Agent's Hands)

State of the art: All strong systems converge on the same toolset: shell, file ops, browser automation, search, MCP for extensibility. OpenHands' CodeAct (Python as action primitive) is the most expressive. Claude Code's built-in toolset is the most ergonomically refined. Hermes's 60+ tools with 6 terminal backends and approval guardrails is the widest production-proven set. Velhari's per-worker guardrail system (path scoping + command blocking + NATS audit) is the best safety architecture.

Best implementations: Hermes for breadth. Claude Code for ergonomics. Velhari for safety architecture. OpenHands for CodeAct expressiveness.

Known pitfalls: (1) Too many tools = token waste. (2) No per-task tool scoping — Claude Code's `allowed_tools` per subagent is the right model. (3) Tools without guardrails.

BORROW THIS / FIX THAT: Claude Code's `allowed_tools` per-agent scoping. Velhari's guardrail factory (path scoping + command blocking + audit per tool call). Hermes's approval pipeline for dangerous tools. OpenHands' SecurityAnalyzer action-scoring (LOW/MEDIUM/HIGH/UNKNOWN). MCP for all external tools — don't build custom wrappers. For token efficiency: use CodeAct-style Python generation for complex multi-step operations rather than composing JSON tool call sequences.

C6: Skills (Reusable Packaged Capabilities)

State of the art: Hermes has the most rigorous skill system: agentskills.io standard, conditional activation gates, multi-source hubs, agent auto-creation after 5+ tool call tasks, security scanner, versioning, `/journey` timeline. OpenClaw is close with adversarial self-extension and ClawHub's 5,700+ community skills. Letta Skill Learning (Dec 2025) adds agent-authored packaging — newest but least battle-tested.

Best implementations: Hermes — agent auto-creates skills, conditional activation, security scanning, multi-source distribution. OpenClaw second — widest community ecosystem, strongest adversarial self-extension pattern.

Known pitfalls: (1) Skills without self-authoring are just macros (Claude Code, Goose, OpenHands, CrewAI). (2) Loading all skills every session wastes tokens. (3) Agent-created skills without security scanning can contain exfiltration logic. (4) Skills without deduplication bloat over time.

BORROW THIS / FIX THAT: agentskills.io SKILL.md format with YAML frontmatter. Hermes's trajectory-to-skill pipeline: after N-tool-call task, package it as a SKILL.md. Progressive disclosure (metadata only until activated). Conditional activation gates. Security scanner for agent-created skills. Multi-scope storage (bundled/user/project). `/journey` skill timeline. Add a deduplication pass before skill creation: check semantic similarity against existing skills, merge if above threshold (same logic as CM's memory merge).

C7: Interface / Gateway

State of the art: Claude Code has the widest cross-surface coverage (terminal/IDE/desktop/web/Slack/GitHub/CI-CD/channels/mobile). Hermes has the widest messaging platform coverage (22 platforms + OpenAI proxy endpoint for ecosystem

reuse). OpenClaw adds voice and Canvas (visual workspace). Letta ADE is the best *developer inspection* interface. Manus's screen mirror + shareable replay is the best *end-user explainability* interface.

Best implementations: Claude Code for surface breadth. Hermes for platform breadth. Letta ADE for developer inspection. OpenClaw for personal-use completeness.

Known pitfalls: (1) Interface as afterthought (headless libraries). (2) Cloud-only violates local-first principle. (3) No reasoning trace in UI — most interfaces show *what* the agent did but few show *why*.

BORROW THIS / FIX THAT: Claude Code's surface-sharing model (same agent loop, multiple surfaces). Hermes's *hermes proxy* (expose as OpenAI-compatible endpoint for ecosystem reuse). Letta ADE's three simulation modes (Debug/Interactive/Simple). OpenClaw's voice interface. **Priority order for Second Brain:** Slack (primary work surface) → web app (admin/review) → terminal (Kaushik development) → mobile (ambient access). All surfaces must share the same CM memory + SOUL.md identity.

C8: Self-Correction

State of the art: Claude Code's hooks system is the most mechanically rigorous: deterministic behavioral control at every lifecycle event with *additionalContext* injection, *updatedInput* modification, *decision*: "block" veto — all independent of model reasoning. OpenHands' StuckDetector is the best *within-session* loop detection (5 semantic pathology patterns, error taxonomy). Hermes's trajectory-to-skill pipeline + completion contracts is the best *cross-session behavioral change* mechanism (the "skill on disk" is verifiable proof of learning). OpenClaw's adversarial self-extension + HEARTBEAT promotion is the best *scheduled consolidation* pattern.

Best implementations: Claude Code for decision-point control. Hermes for verifiable cross-session learning. OpenClaw for scheduled identity consolidation.

Known pitfalls: (1) Self-correction without persistence — within-session correction is lost between sessions. (2) Hooks without self-generation — require human authoring, don't auto-emerge from observed failures. (3) Memory-based reflection is indirect — having a memory of past failures doesn't guarantee avoidance.

BORROW THIS / FIX THAT: Claude Code's hooks architecture for decision-point control (PreToolUse veto, PostToolUse context injection). Hermes's trajectory-to-skill pipeline for verifiable cross-session learning. OpenClaw's HEARTBEAT.md scheduled pass for consolidation. OpenHands' StuckDetector (5 semantic pathology patterns) for in-session loop detection. CM stores the lessons; hooks deliver them at the decision point — this is the requirement's key distinction: *memory stores, self-correction changes what the agent does*. Build the gap: when a hook blocks or corrects an action, store that event to CM as an episodic memory → surface past correction patterns before similar actions in future sessions.

C9: Observability / Trust

State of the art: Letta's Step-level telemetry (nanosecond precision, OTel export, ClickHouse, ADE glass-box) is the most technically rigorous. OpenHands' immutable EventLog (audit trail by construction, SecurityAnalyzer action scoring, SecretRegistry) is the most architecturally sound. Hermes's dual-path observability (Langfuse + OTel, structured log separation, */journey* memory graph) is the most operationally complete. OpenClaw's ClawMetry dashboard (purpose-built agent audit) is the most product-complete.

Best implementations: Letta for technical depth. OpenHands for architectural correctness (immutability by construction). Hermes for operational completeness. Letta ADE for interactive glass-box inspection.

Known pitfalls: (1) "Observable if you build the observer" (Claude Code) — leaving all observability to hooks means every deployment reinvents the stack. (2) Cloud-hosted reasoning traces (CrewAI AMP) violate local-first and privacy requirements. (3) Screen-mirror without structured trace (Manus) is good for end-users, useless for engineering audit.

BORROW THIS / FIX THAT: Letta's *Step* record model — every inference generates a structured record: model, tokens (prompt/completion/cached/reasoning), tool calls, latency (ns), trace_id. Store these locally (SQLite or SurrealDB alongside CM). OpenHands' immutable append-only event log — reasoning trace is an append-only artifact, never edited. Hermes's */journey* concept — a user-facing timeline of what the agent has done, learned, and built. The hard requirement is *retroactive veto*: design from day one so every autonomous action is reversible or has an undo plan stored in the trace.

Part 4 — Sources

#	URL	System
1	https://github.com/openclaw/openclaw	OpenClaw
2	https://docs.openclaw.ai/concepts/memory	OpenClaw
3	https://softmaxdata.com/blog/deep-dive-into-openclaws-agentic-orchestrate-design-patterns-philosophy-framework-choices/	OpenClaw
4	https://velvetshark.com/openclaw-memory-masterclass	OpenClaw
5	https://chatinfo.medium.com/openclaws-self-improvement-loop-letting-an-ai-change-its-own-code-under-supervision-0cf5ef76a94a	OpenClaw
6	https://www.stack-junkie.com/blog/openclaw-learning-without-rl	OpenClaw
7	https://github.com/VoltAgent/awesome-openclaw-skills	OpenClaw
8	https://github.com/NousResearch/hermes-agent	Hermes
9	https://deepwiki.com/NousResearch/hermes-agent/1.1-architecture-overview	Hermes
10	https://blakecrosley.com/guides/hermes	Hermes
11	https://kisztof.medium.com/hermes-agent-review-nous-researchs-self-improving-ai-agent-e72bc244435a	Hermes
12	https://dev.to/observabilityguy/put-a-microscope-on-hermes-full-visibility-into-agent-execution-2b1a	Hermes
13	https://betterstack.com/community/guides/ai/hermes-agent/	Hermes
14	https://code.claude.com/docs/en/overview	Claude Code
15	https://code.claude.com/docs/en/hooks	Claude Code
16	https://code.claude.com/docs/en/agent-teams	Claude Code
17	https://code.claude.com/docs/en/skills	Claude Code
18	https://code.claude.com/docs/en/sub-agents	Claude Code
19	https://code.claude.com/docs/en/memory	Claude Code
20	https://platform.claude.com/docs/en/agent-sdk/hooks	Claude Code
21	https://github.com/disler/claude-code-hooks-multi-agent-observability	Claude Code
22	https://www.ksred.com/the-claude-agent-sdk-what-it-is-and-why-its-worth-understanding/	Claude Code
23	https://arxiv.org/abs/2310.08560	Letta/MemGPT
24	https://github.com/letta-ai/letta	Letta/MemGPT
25	https://docs.letta.com/guides/agents/memory-blocks	Letta/MemGPT
26	https://deepwiki.com/letta-ai/letta/3-memory-system	Letta/MemGPT
27	https://www.letta.com/blog/letta-v1-agent	Letta/MemGPT
28	https://www.letta.com/blog/memory-blocks/	Letta/MemGPT
29	https://docs.letta.com/guides/agents/architectures/sleeptime/	Letta/MemGPT
30	https://deepwiki.com/letta-ai/letta/13.1-telemetry-and-monitoring	Letta/MemGPT
31	https://www.letta.com/blog/context-bench-skills/	Letta/MemGPT
32	https://docs.letta.com/guides/core-concepts/tools/mcp-tools	Letta/MemGPT
33	https://arxiv.org/html/2511.03690v1	OpenHands
34	https://deepwiki.com/All-Hands-AI/OpenHands	OpenHands

#	URL	System
35	https://docs.openhands.dev/sdk/arch/overview	OpenHands
36	https://www.openhands.dev/blog/openhands-meets-slack-ai-powered-development-in-your-workspace	OpenHands
37	https://deepwiki.com/OpenHands/software-agent-sdk/3.3-sub-agent-delegation-and-task-management	OpenHands
38	https://pablordoricaw.github.io/multi-agent-systems-research/deep-dives/openhands/	OpenHands
39	https://deepwiki.com/block/goose	Goose
40	https://github.com/block/goose	Goose
41	https://goose-docs.ai/docs/mcp/memory-mcp/	Goose
42	https://github.com/block/goose/discussions/4389	Goose
43	https://langfuse.com/docs/integrations/goose	Goose
44	https://docs.crewai.com/concepts/memory	CrewAI
45	https://docs.crewai.com/concepts/crews	CrewAI
46	https://docs.crewai.com/concepts/flows	CrewAI
47	https://sparkco.ai/blog/deep-dive-into-crewai-memory-systems	CrewAI
48	https://docs.crewai.com/en/observability/tracing	CrewAI
49	https://signoz.io/docs/crewai-observability/	CrewAI
50	https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus	Manus
51	https://arxiv.org/html/2505.02024v1	Manus
52	https://manus.im/blog/manus-skills	Manus
53	https://manus.im/docs/integrations/mcp-connectors	Manus
54	https://manus.im/blog/projects-connectors	Manus
55	https://www.zenml.io/llmops-database/building-an-ai-agent-platform-with-cloud-based-virtual-machines-and-extended-context	Manus
56	C:/Projects/cognitive-memory/README.md	CM (first-hand)
57	C:/Projects/cognitive-memory/src/cognitive_memory/engine.py	CM (first-hand)
58	C:/Projects/cognitive-memory/src/cognitive_memory/retrieval.py	CM (first-hand)
59	C:/Projects/cognitive-memory/src/cognitive_memory/decay.py	CM (first-hand)
60	C:/Projects/cognitive-memory/src/cognitive_memory/consolidation.py	CM (first-hand)
61	C:/Projects/cognitive-memory/src/cognitive_memory/models.py	CM (first-hand)
62	C:/Projects/velhari/src/velhari/orchestrator.py	Velhari (first-hand)
63	C:/Projects/velhari/src/velhari/guardrails.py	Velhari (first-hand)
64	C:/Projects/velhari/src/velhari/models.py	Velhari (first-hand)
65	Velasari: Claude Code global settings + CLAUDE.md (read directly)	Velasari (first-hand)

